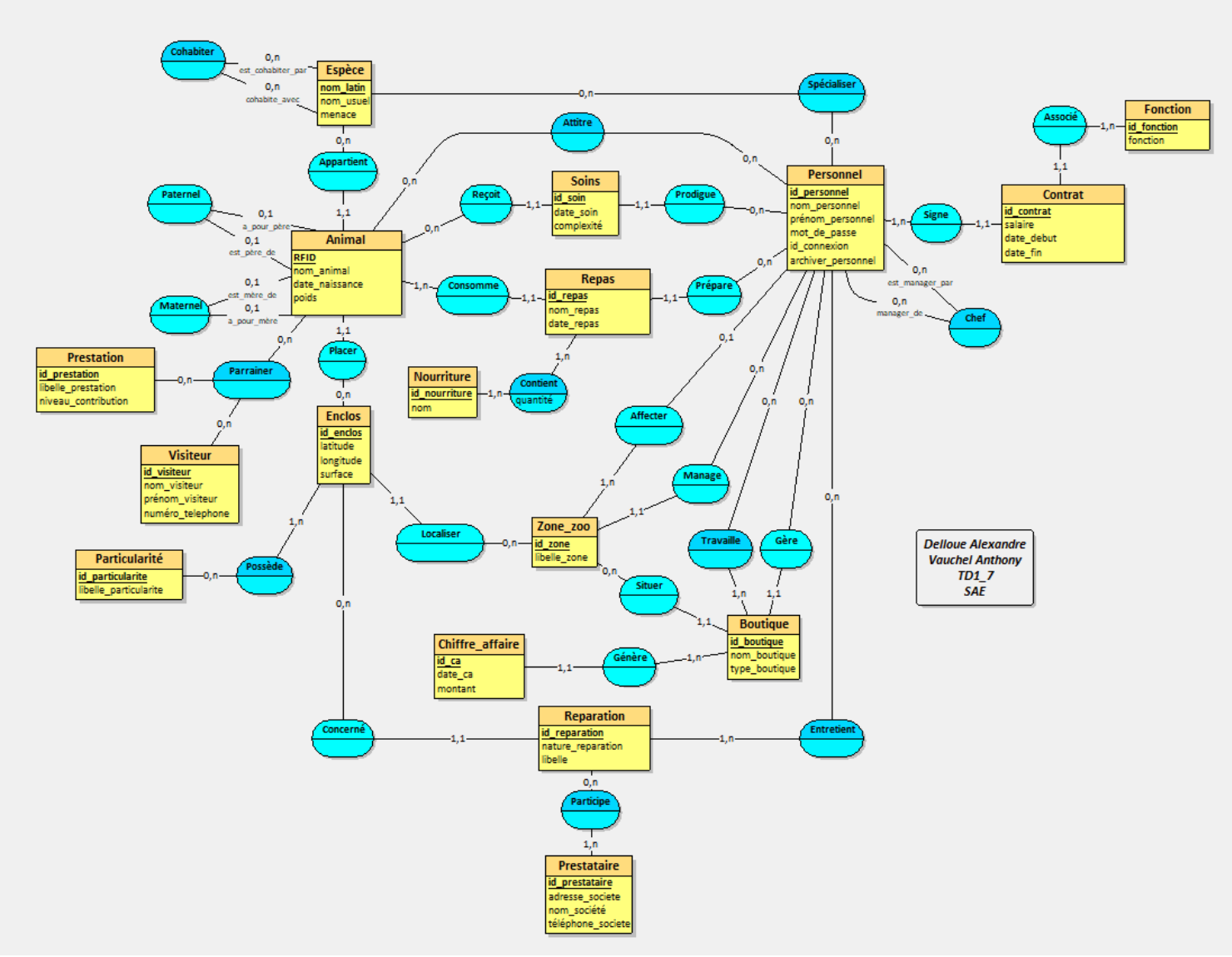
Rapport BD

Zoo’land 2026

Sommaire

MCD..... 3
Justification du MCD..... 4
Dictionnaire de données..... 6
Schéma relationnel..... 9
Requête compatible Oracle et schéma algébrique..... 10

MCD


Cohabiter
0,n
est_cohabiter_par
0,n
cohabite_avec
Espèce
nom_latin
nom_usuel
menace
0,n
Spécialiser
Attitre
Associé
1,n
Fonction
id_fonction
fonction
0,n
Appartient
1,1
Personnel
id_personnel
nom_personnel
prénom_personnel
mot_de_passe
id_connexion
archiver_personnel
1,1
Contrat
id_contrat
salaire
date_debut
date_fin
Paternel
0,1
a_pour_père
0,1
est_père_de
Animal
RFID
nom_animal
date_naissance
poids
Reçoit
1,1
Soins
id_soin
date_soin
complexité
1,1
Prodigue
0,n
1,n
Signe
1,1
0,n
est_manager_par
0,n
manager_de
Chef
Maternel
0,1
est_mère_de
0,1
a_pour_mère
1,n
Consomme
1,1
Repas
id_repas
nom_repas
date_repas
1,1
Prépare
Prestation
id_prestation
libelle_prestation
niveau_contribution
0,n
Parrainer
1,1
Placer
0,n
Nourriture
id_nourriture
nom
1,n
Contient
quantité
1,n
Enclos
id_enclos
latitude
longitude
surface
Affecter
Visiteur
id_visiteur
nom_visiteur
prénom_visiteur
numéro_telephone
Manage
1,1
1,n
Localiser
0,n
Zone_zoo
id_zone
libelle_zone
Travaille
Gère
Particularité
id_particularite
libelle_particularite
0,n
Possède
Situer
1,n
1,1
Boutique
id_boutique
nom_boutique
type_boutique
Chiffre_affaire
id_ca
date_ca
montant
1,1
Génère
1,n
Delloue Alexandre
Vauchel Anthony
TD1_7
SAE
Concerné
1,1
Reparation
id_reparation
nature_reparation
libelle
1,n
Entretient
Participe
1,n
Prestataire
id_prestataire
adresse_societe
nom_société
téléphone_societe

Justification du MCD

Dans le cadre de cette SAÉ, nous avons conçu un MCD structurant la base de données du zoo Zoo'Land. Celle-ci couvre quatre domaines fonctionnels : le suivi des animaux (soins, repas), la gestion du personnel, l'architecture des installations, et les interactions visiteurs-animaux. Les choix présentés ci-dessous concernent uniquement les décisions prises en autonomie, à l'exclusion des contraintes explicitement imposées par le sujet.

1. La gestion des animaux

La filiation est modélisée par deux relations réflexives distinctes sur l'entité Animal, afin d'imposer un maximum de deux parents — ce qu'une relation réflexive unique en (0,n) ne permettrait pas. Chaque animal est obligatoirement rattaché à une espèce, laquelle dispose d'un attribut binaire menacé (oui/non). La cohabitation inter-espèces est gérée par une relation réflexive sur Espèce en (0,n)-(0,n), les informations nécessaires étant déjà portées par l'entité. Chaque espèce doit posséder au moins un soigneur expert : nous considérons qu'un zoo intégrant une nouvelle espèce prépare préalablement un expert, et les départs de personnel n'entraînent pas de suppression (voir gestion du personnel).
Chaque animal dispose d'un soigneur attitré via la relation Attitré, maillée avec Personnel. Les soigneurs remplaçants ne font l'objet d'aucune entité dédiée : tout soigneur ayant prodigué des soins sans être attitré est implicitement considéré comme remplaçant.
Les soins sont représentés par l'entité Soins : un animal en bonne santé peut n'en recevoir aucun, mais tout soin créé doit être affecté à un animal. La préparation et l'administration sont considérées comme un processus unique, réalisé par un seul soigneur. La même logique s'applique aux repas : l'entité Repas assure le suivi de la consommation, tandis que l'entité Nourriture gère la composition, via l'association Contient qui enregistre les quantités. Cela permet une traçabilité complète, utile notamment en cas d'erreur d'un soigneur.

2. Gestion du personnel

En l'absence de hiérarchie entre entités, l'ensemble du personnel est regroupé dans une entité unique Personnel, associée à une entité Contrat portant la fonction occupée. Les cardinalités minimales à 0 permettent de gérer l'absence temporaire d'affectation pour certains rôles (ex. : entretien). La cohérence des affectations — éviter qu'un soigneur soit assigné à une boutique — sera gérée dans le site web ou en SQL.
La fonction est volontairement stockée dans le contrat plutôt que directement dans Personnel, afin d'éviter toute redondance et de conserver un historique : la fonction courante d'un agent se déduit de son contrat le plus récent (date de fin absente ou postérieure à la date du jour). La hiérarchie entre soigneurs est modélisée par une relation réflexive sur Personnel. Enfin, la gestion des boutiques distingue les associations Travaille et Gère, séparant employés et directeurs pour une administration plus fine.

3. Architecture du zoo

Les animaux sont placés dans des enclos, chacun localisé dans une zone unique, structurant ainsi le zoo en secteurs géographiques. Les caractéristiques des enclos sont externalisées dans une entité dédiée, reliée par une association maillée : cela facilite l'ajout ou le retrait de particularités sans modifier la structure principale, tout en conservant l'historique des caractéristiques supprimées pour une éventuelle réutilisation. Les réparations sont liées à des prestataires via une association maillée, permettant d'associer plusieurs prestataires à une même réparation (ex. : remplacement et peinture de barrières par deux entreprises distinctes). Enfin, on autorise à ajouter des prestataires sans qu'ils aient participé à des réparations : dans le cas d'une collaboration future par exemple.

4. Interaction avec les visiteurs

Les visiteurs sont identifiés par leurs nom, prénom, téléphone et adresse e-mail. Les prestations sont gérées via une association ternaire maillée reliant Visiteur, Prestation et Animal : cette structure découple les trois entités tout en maintenant leurs liens via l'association Parrainer. Elle permet à un visiteur d'accéder à plusieurs prestations portant sur des animaux différents, sans dépendance entre les entités.

Dictionnaire de données

Attribut	Description	Type	Longueur max	Propriété
nom_latin	Nom en latin de l'espèce	Texte	50	Clé
nom_usuel	Nom usuel de l'espèce	Texte	50	
menace	Espèce menacé ou non	Caractère	1	
id_particularité	Identifiant	Entier		Clé
libelle_particularité	Nom de la particularité	Texte	50	
id_prestataire	Identifiant	Entier		Clé
adresse_societe	Adresse du prestataire	Texte	50	
nom_societe	Nom de la société du prestataire	Texte	50	
telephone_societe	Numéro de téléphone de la société	Entier	10	
id_visiteur	Identifiant	Entier		Clé
nom_visiteur	Nom du visiteur	Texte	50	
prenom_visiteur	Prénom du visiteur	Texte	50	
numero_telephone	Numéro de téléphone du visiteur	Entier	10	
id_prestation	Identifiant	Entier		Clé
libelle_prestation	Nom de la prestation	Texte	50	
niveau_contribution	Bronze, Argent ou Or	Texte	6	
id_fonction	Identifiant	Entier		Clé
fonction	Nom de la fonction	Texte	50	
id_nourriture	Identifiant	Entier		Clé
nom_nourriture	Nom de la nourriture	Texte	50	
RFID	Identifiant	Entier		Clé
nom_animal	Nom de l'animal	Texte	50	
date_naissance	Date de naissance de l'animal	Date		

poids	Poids de l'animal	Réel	4,2	
RFID_a_pour_pere	RFID du père de l'animal	Entier		
RFID_a_pour_mere	RFID de la mère de l'animal	Entier		
id_enclos	Identifiant	Entier		Clé
latitude	Latitude de l'enclos	Réel		
longitude	Longitude de l'enclos	Réel		
surface	Surface de l'enclos	Réel		
id_zone	Identifiant	Entier		Clé
libelle_zone	Nom de la zone	Texte	50	
id_personnel	Identifiant	Entier		Clé
nom_personnel	Nom du personnel	Texte	50	
prenom_personnel	Prénom du personnel	Texte	50	
mot_de_passe	Mot de passe de connexion du personnel	Texte	255	Hash
id_connexion	Identifiant de connexion du personnel	Texte	100	
archiver_personnel	O ou U si le personnel est archivé	Caractère	1	
id_boutique	Identifiant	Entier		Clé
nom_boutique	Nom de la boutique	Texte	50	
type_boutique	Type de boutique	Texte	50	
id_ca	Identifiant	Entier		Clé
date_ca	Date du chiffre d'affaire	Date		
Montant	Montant du chiffre d'affaire	Réel		
id_reparation	Identifiant	Entier		Clé
nature_reparation	Nature de la réparation	Texte	50	

libelle_reparation	Description ou nom de la réparation	Texte	50	
id_soin	Identifiant	Entier		Clé
date_soin	Date d’administration du soin	Date		
complexité	Complexité du soin	Texte	50	
id_repas	Identifiant	Entier		Clé
nom_repas	Nom du repas	Texte	50	
date_repas	Date d’administration du repas	Texte	50	
id_contrat	Identifiant	Entier		Clé
salaire	Salaire associé au contrat	Réel	10, 2	
date_debut	Date de début du contrat	Date		
date_fin	Date de fin du contrat	Date		
quantité	Quantité de nourriture associé à un repas	Entier		

Schéma relationnel

ESPÈCE(**nom_latin**, nom_usuel, menace);
COHABITER(**#nom_latin_est_cohabiter_par, #nom_latin_cohabiter_avec**);
ANIMAUX(**RFID**, nom_animal, date_naissance, poids, #RFID_a_pour_pere, #RFID_a_pour_mere, #id_enclos, #nom_latin);

VISITEUR(**id_visiteur**, nom_visiteur, prenom_visiteur, numero_telephone);
PRESTATIONS(**id_prestation**, libelle_prestation, niveau_contribution);
PARRAINER(**#RFID, #id_visiteur, #id_prestation**);

ENCLOS(**id_enclos**, latitude, longitude, surface, #id_zone);
ZONE_ZOO(**id_zone**, libelle_zone, #id_personnel);
POSSÈDE(**#id_enclos, #libelle_particularite**);
PARTICULARITE(**id_particularite**, libelle_particularite);
BOUTIQUE(**id_boutique**, nom_boutique, type_boutique, #id_zone, #id_personnel);
TRAVAILLE(**#id_personnel, #id_boutique**);
CHIFFRE_AFFAIRE(**id_ca**, date_ca, montant, #id_boutique);

REPARATION(**id_reparation**, nature_reparation, libelle, #id_enclos);
PRESTATAIRE(**id_prestataire**, adresse_societe, nom_societe, telephone_societe);
PARTICIPE(**#id_prestataire, #id_reparation**);
ENTRETIENT(**#id_personnel, #id_reparation**);

PERSONNEL(**id_personnel**, nom_personnel, prenom_personnel, mot_de_passe, id_connexion, archiver_personnel, #id_zone);
CHEF(**#id_personnel_manager_de, #id_personnel_est_manager_par**);
FONCTION(**id_fonction**, fonction);
CONTRAT(**id_contrat**, salaire, date_debut, date_fin, #id_fonction, #id_personnel);

SPÉCIALISER(**#nom_latin, #id_personnel**);
ATTITRÉ(**#RFID, #id_personnel**);
SOINS(**id_soin**, date_soin, complexité, #id_personnel, #RFID);
REPAS(**id_repas**, nom_repas, date_repas, #RFID, #id_personnel);
CONTIENT(**#id_repas, #id_nourriture**, quantite);
NOURRITURE(**id_nourriture**, nom);

Requête compatible Oracle et schéma algébrique

1. *Tracer l'historique des fonctions, en affichant la date de début et de fin, de l'employé ayant comme nom Sorton, par ordre chronologique*

```
SELECT fonction, TO_CHAR(date_debut, 'DD/MM/YYYY'), TO_CHAR(date_fin,
'DD/MM/YYYY')
FROM Personnel p, Contrat c, Fonction f
WHERE p.id_personnel = c.id_personnel AND c.id_fonction = f.id_fonction AND
nom_personnel='Sorton'
ORDER BY date_debut;
```

$$\pi_{fonction,\, date_debut,\, date_fin}\,(\sigma_{nom_personnel='Sorton'}(Fonction \bowtie (Personnel \bowtie Contrat)))$$

2. *Quels sont le(s) nom(s) de(s) soigneur(s) ayant soigné tous les animaux ?*

Cette question demandant une division, nous allons réaliser les trois manières de faire une division en langage sql

↪ 1ère façon : HAVING / GROUP BY

```
SELECT nom_personnel, prenom_personnel
FROM Personnel p, Soins s
WHERE p.id_personnel = s.id_personnel
GROUP BY p.id_personnel, p.nom_personnel, p.prenom_personnel
HAVING COUNT(DISTINCT s.RFID) = (SELECT COUNT(*) FROM Animal);
```

↪ 2ème façon : double NOT EXISTS

```
SELECT DISTINCT nom_personnel, prenom_personnel
FROM Personnel p
WHERE NOT EXISTS (
    SELECT a.RFID FROM Animal a
    WHERE NOT EXISTS (
        SELECT * FROM Soins s
        WHERE s.id_personnel = p.id_personnel
        AND s.RFID = a.RFID
    )
);
```

↪ 3ème façon : MINUS

```
SELECT DISTINCT nom_personnel, prenom_personnel
FROM Personnel p
WHERE NOT EXISTS (
    SELECT a.RFID FROM Animal a
    MINUS
    SELECT s.RFID FROM Soins s
    WHERE s.id_personnel = p.id_personnel
);
```

$$\pi_{nom_personnel,\, prenom_personnel}((\pi_{id_personne,\, RFID}(Soins) \div \pi_{RFID}(Animal)) \bowtie Personnel)$$

3. *Lister les soins (id_soin, date_soin, complexité) réalisés entre le 05/02/2025 et le 02/03/2025, triés par complexité décroissante*

```
SELECT id_soin, date_soin, complexite
FROM Soins
WHERE date_soin >= TO_DATE('05022025', 'DDMMYYYY') AND date_soin <=
TO_DATE('02032025', 'DDMMYYYY')
ORDER BY complexite DESC;
```

$$\pi_{id_soin,\, date_soin,\, complexite}(\sigma_{date_soin>=05/02/2025\ AND\ date_soin<=02/03/2025}(Soins)$$

4. *Pour chaque enclos, afficher son identifiant et le nombre d'animaux qu'il contient*

```
SELECT e.id_enclos, COUNT(a.RFID) as NombreAnimal
FROM Enclos e, Animal a
WHERE e.id_enclos = a.id_enclos
GROUP BY e.id_enclos;
```

$$Compte_{id_enclos}(Enclos \bowtie Animal,\ RFID)$$

5. *Quels sont les enclos qui contiennent en moyenne des animaux de plus de 50 kg ? Afficher l'id de l'enclos et le poids moyen*

```
SELECT e.id_enclos, AVG(poids)
FROM Enclos e, Animal a
WHERE e.id_enclos = a.id_enclos
GROUP BY e.id_enclos
HAVING AVG(poids) > 50;
```

$$\pi_{id_enclos,\, Moyenne}(\sigma_{Moyenne>50}(Moyenne_{id_enclos}(Enclos \bowtie Animal,\ poids))$$

6. *Quel est l'animal (nom) ayant le poids le plus élevé dans le zoo ?*

```
SELECT nom_animal
FROM Animal
WHERE poids = (SELECT MAX(poids) FROM Animal);
```

$$\pi_{nom_animal}(\sigma_{poids=Max}(Animal \times Max(Animal,\ poids))$$

7. *Quels sont les animaux (nom) qui ont reçu des soins ET qui ont consommé au moins un repas ?*

```
SELECT DISTINCT a.nom_animal
FROM Animal a
WHERE a.RFID IN (
    SELECT s.RFID
    FROM Soins s
    INTERSECT
    SELECT c.RFID
    FROM Consomme c
);
```

$$\pi_{nom_animal}(Animal \bowtie (\pi_{RFID}(Soins) \cap \pi_{RFID}(Consomme)))$$

8. *Créer une vue BILAN-ANIMAL qui, pour chaque animal, affiche son nom, le nombre de soins reçus et la date du dernier soin :*

```
CREATE VIEW BILAN_ANIMAL AS
SELECT a.nom_animal, COUNT(s.id_soin) as nb_soin, MAX(s.date_soin) as
dernier_soin
FROM Animal a, Soins s
WHERE a.RFID = s.RFID
GROUP BY a.nom_animal;
```

9. *A partir de cette vue, affiche uniquement les animaux ayant reçu plus de 3 soins, triés par date de dernier soin décroissante*

```
SELECT nom_animal, nb_soin, dernier_soin
FROM BILAN_ANIMAL
WHERE nb_soin > 3
ORDER BY dernier_soin DESC;
```

$$\pi_{nom_animal,\, nb_soin,\, dernier_soin}(\sigma_{nb_soin > 3}(BILAN_ANIMAL))$$

10. *Afficher le nom des enclos, leur surface et les réparations associées si elles existent*

```
SELECT e.id_enclos, e.surface, r.id_reparation, r.nature_reparation
FROM ENCLOS e
LEFT JOIN REPARATION r ON e.id_enclos = r.id_enclos;
```

$$\pi_{id_enclos,\, surface,\, id_reparation,\, nature_reparation}(Enclos \bowtie Reparation)$$

11. *Afficher les prestations et les id des visiteurs qui sont associés à cette prestation*

```
SELECT p.id_prestation, p.libelle_prestation, pa.id_visiteur
FROM PARRAINER pa
RIGHT JOIN PRESTATIONS p ON pa.id_prestation = p.id_prestation
ORDER BY p.id_prestation;
```

$$\pi_{id_prestation,\, libelle_prestation,\, id_visiteur}(Parrainer \rhd\lhd| \; Prestations)$$

12. *Afficher, sous forme de couple, les animaux vivant dans le même enclos, ainsi que l'id de l'enclos en question*

```
SELECT a1.RFID AS animal1, a2.RFID AS animal2, a1.id_enclos
FROM Animal a1, Animal a2
WHERE a1.id_enclos = a2.id_enclos
  AND a1.RFID < a2.RFID;
```

$$\pi_{RFID_animal1,\, RFID_animal2}(\sigma_{RFID_animal1 < RFID_animal2 \,\wedge\, id_enclos_animal1 = id_enclos_animal2}(Animal \times Animal))$$